# A LOW-COST WORKFLOW FOR DETECTING AND VISUALISING SUSPICIOUS NETWORK BEHAVIOUR IN SMART-HOME IOT DEVICES



Cybersecurity Challenge Studio Industry Stream

Zhenfu Shi, Wenxuan Jin, Ruoyang Ma

# Table of Contents


.....0

1 Executive Summary .....4

2 Industry Problem and Real-World Context .....5

2.1 Problem Statement .....5

2.2 Where the Problem Occurs .....5

2.3 Who is Affected .....5

2.4 Consequences if the Problem is Not Addressed .....6

2.5 Illustrative Incident .....6

2.6 Why the Problem Is Worth Solving .....8

3 Market, User, or Stakeholder Analysis .....9

3.1 Stakeholder Summary .....9

4 Existing Solutions and Gap Analysis .....10

4.1 Existing Solutions .....10

4.2 Gap Analysis .....11

5 Product / Project Scope .....12

5.1 In scope .....12

5.2 Out of Scope .....13

5.3 Assumptions and Constraints .....13

6 Requirements and Success Criteria .....14

6.1 Functional Requirements .....14

6.2 Non-Functional Requirements .....14

6.3 Success Criteria .....15

7 Proposed Solution and Technical Workflow .....16

7.1 Overview of the Proposed Solution .....16

7.2 Stage One: Passive Traffic Capture .....16

7.3 Stage Two: Device Inventory and Baseline Construction .....16

7.4 Stage Three: Network Segmentation .....16

7.5 Stage Four: Before-and-After Comparison ..... 16
7.6 Stage Five: Evidence Compilation and Reporting ..... 17
8 System Architecture or Implementation Design ..... 18
8.1 Flat network before segmentation ..... 18
8.2 Segmented Network (State C) ..... 19
9 Security, Privacy, and Risk Analysis ..... 21
9.1 Privacy Risk to the Household ..... 21
9.2 Risk to the Availability and Integrity of the Environment Under Test ..... 21
9.3 Risks Identified in the Devices Under Study ..... 21
9.4 Evidence Handling ..... 22
9.5 Authorisation Boundary and Third-Party Risk ..... 22
10 Development Progress and Evidence ..... 23
10.1 Capture Script and Methodology ..... 23
10.2 Baseline Capture Run Log ..... 24
10.3 Device Inventory ..... 24
10.4 Segmentation Configuration ..... 24
10.5 Troubleshooting Record: Connection-Tracking False Positive ..... 25
10.6 Troubleshooting: Hardware Flow Offload and Capture Visibility ..... 27
11 Testing, Validation, and Evaluation ..... 28
11.1 Evaluation Approach ..... 28
11.2 Capture and Baseline Validation ..... 28
11.3 Detection and Mitigation Results ..... 28
11.4 Metrics and Remaining Evaluation ..... 29
12 Deployment, Operation, and Maintenance Considerations ..... 30
12.1 Deployment Environment and Permissions ..... 30
12.2 Deployment and Recovery Procedure ..... 30
12.3 Monitoring and Maintenance ..... 30
12.4 Documentation and Operational Feasibility ..... 31
13 Limitations and Future Improvement ..... 32

13.1 Dataset and Generalisation ..... 32
13.2 Visibility and Experimental Constraints ..... 32
13.3 Detection and Usability Limitations ..... 32
14 Conclusion ..... 33
15 References and Appendices ..... 34

# 1 Executive Summary

Personal computers and storage devices are frequently connected to a network with smart-home devices. Most people cannot see where those devices are connecting and whether they've undergone changes in their behaviour. If the device is compromised, it can use such access to probe other systems on such a flat network.

We constructed and tested an inexpensive workflow using router captures, Zeek/py analysis, per-routing traffic baselines, changes in VLANs and Firewall and a splunk dashboard. Recorded ~24 hrs of traffic for each of the flat and segmented states; developed baselines for five devices. Another Mininet lab offers a reproducible setting to study the impact of changing the network policy on network exposure.

Transparent rules uncovered the three categories of simulated attacks: lateral scanning, DNS bypass and data upload. Upload test was performed including bulk and slow variants, which were both found. The last router setup denied access to the unauthorised paths which had been tried from the IoT network. All 28 documented device-function checks were successful, which included the cases of local and remote device control and recovery after router restart. One plug had to be power cycled following a wireless restart.

A sanitised set of captures, scripts, configuration evidence, test outputs and dashboard guide are included in the portfolio. These are the results achieved with the workflow within this home.

# 2 Industry Problem and Real-World Context

## 2.1 Problem Statement

Consumer and small-office IoT devices such as cameras, smart plugs, speakers, hubs, thermostats are being deployed at scale but are frequently insecure. Manufacturers often prioritise cost and time to market over security. Owners cannot install antivirus, endpoint detection or logging agents on most IoT devices. The device is effectively a black box, and the owner can only judge its health by whether it appears to work. Network traffic is therefore often the only practical source of visibility, and in most homes and small organisations it is not monitored. A hijacked camera or thermostat usually continues to work normally while carrying out malicious activity in the background. The user sees no error or warning, so detection depends entirely on someone noticing abnormal network behaviour.

## 2.2 Where the Problem Occurs

- Home networks: Most home routers place all devices on a single flat network. Smart TVs, cameras, laptops and phones can therefore reach each other, and a compromise of the weakest IoT device exposes everything else on the network.

- Small and home offices: Small businesses often don't segregate IoT devices from work computers and have no dedicated IT or security staff. There is usually no inventory of devices, no monitoring, and no incident response process.

- Remote-work environments: Corporate laptops now regularly connect through home networks that the employer does not control. A compromised home device can become an indirect route towards corporate resources. For example, through traffic interception on the local network.

## 2.3 Who is Affected

- Householders: They face loss of privacy through camera and microphone exposure or behavioural profiling. Their devices can also be recruited into botnets without their knowledge, leaving them unaware they are contributing to attacks on others.

- Small businesses and employers: A compromised device gives an attacker a foothold inside the network from which to move laterally towards more valuable

systems such as file shares and business applications. The business often lacks the tooling to detect this movement.

- Security teams and service providers: Analysts and IT support providers have little device-level telemetry, so it is hard to triage an incident or say with confidence whether an IoT device was involved.

- Manufacturers and regulators: Vendors face growing pressure to demonstrate baseline security, and regulators need evidence of how devices behave in practice to justify and enforce standards.

- The internet: Compromised devices are commonly used to attack third parties, so the harm extends beyond the owner to service providers and their customers.

## 2.4 Consequences if the Problem is Not Addressed

- Ongoing surveillance or data leakage: An attacker with access to a camera, microphone or hub can collect sensitive information over a long period. Because the device keeps working, the owner may never learn about the exposure.

- Participation in malicious activity: Compromised devices can be used for DDoS, spam or proxying attacker traffic. This can lead to service disruption for others, and to the owner's IP address being blocklisted or blamed.

- Credential theft and lateral movement: An attacker on the same network segment can attempt to capture credentials or scan for vulnerable systems, turning a low-value device compromise into a serious breach.

- Delayed detection and response: Few environments hold a baseline of normal device behaviour, so unusual activity is hard to recognise. Delayed detection increases the time an attacker has and the cost of recovery.

## 2.5 Illustrative Incident

Mirai is the clearest real-world example of how weak IoT devices become a large-scale threat. It was first identified in August 2016 and consists of compromised devices that are controlled remotely by a botmaster. Its defining features are how fast it spread and

how much attack traffic it generated. Between August 2016 and February 2017, it infected more than 600,000 devices.

Mirai uses three main components. The command and control (C&C) server is the link between the botmaster and the botnet, and it passes attack instructions to the bots. The report server keeps a database of newly compromised devices and is updated as bots find victims. The loader receives each victim's address and login credentials from the bots and uses them to log in and install the malware. Separating these roles lets infection and attack run in parallel across a very large number of devices.

Each bot scans random IPv4 addresses for devices with open Telnet ports, targeting port 23 about nine times out of ten and port 2323 the rest of the time. When a bot finds a responsive device, it runs a dictionary attack using a built-in list of factory default username and password. No software vulnerability is needed, only credentials the owner never changed. Successful logins are reported to the report server. The loader then logs in to the victim and instructs it to download the Mirai malware. Once running, the malware shuts down the device's own SSH and Telnet services so that rival malware cannot take it over and then contacts the C&C server. The C&C server specifies the attack type and duration, and the botmaster directs the combined traffic of all bots at one or more targets, typically as a DDoS attack.

One detail of the scanning phase matters for detection. To scan quickly, Mirai bots send TCP SYN packets without completing the three-way handshake, and they set the TCP sequence number equal to the destination IP address. This "Mirai signature" is likely a shortcut that saves memory on resource-constrained devices. It gives defenders a distinctive marker in traffic, and the fast, high volume scanning behaviour is visible on the network even when nothing can be observed on the device itself.

The impact was substantial. Mirai-based attacks included a 623 Gbps flood against security journalist Brian Krebs's website, at the time the largest DDoS attack publicly reported, and an attack of about 1 Tbps against the French hosting provider OVH. In total, about 15,000 DDoS attacks have been linked to Mirai. None of this required sophisticated exploits, only a large population of internet-exposed devices with known default logins (Affinito et al., 2023).

The same behaviour of a compromised device would show up in a home network as anomalies against a device's baseline such as bursts of outbound connection attempts to many unrelated external addresses on ports 23 and 2323. Also contact with addresses the device has never used before and suddenly stops answering on its own management ports.

### 2.6 Why the Problem Is Worth Solving

- The affected population is large: Households and small organisations make up a huge share of IoT deployments, yet they typically have neither security staff nor enterprise tooling.
- Existing solutions are too heavyweight: Enterprise platforms such as SIEMs and IDS tools are powerful but complex and costly for small environments. Consumer products are simple but often don't tell why a device was flagged.
- Skill training: Junior practitioners need practice applying traffic analysis to live, undocumented device behaviour, not only to curated training scenarios.

# 3 Market, User, or Stakeholder Analysis

## 3.1 Stakeholder Summary

| Stakeholder | Key pain points | Limitations | Value from this project |
|---|---|---|---|
| **Home users** | No visibility into what devices do on the network. Cannot tell whether a device is compromised. | Vendor apps report device status, not network behaviour. Flat networks are the default. | Idea of normal device behaviour and a clear segmentation method |
| **Small business and home office owners** | IoT shares a network with work systems. No IT staff. No incident process. | No device inventory, no monitoring, no baseline. Enterprise tools are too costly and complex. | Low cost method to assess IoT risk and contain devices, with evidence to justify changes |
| **Junior SOC analysts** | IoT related alerts arrive with little context. Hard to separate benign from suspicious. | No per device baseline, so false positives and missed anomalies are likely | Worked example of how a baseline improves triage and evidence quality |
| **Penetration testers** | Need a structured way to test IoT segmentation and exposure and to show the result to clients | Findings are often point in time and hard to demonstrate as before and after improvement | Segmentation validation approach with measurable exposure comparison |
| **Privacy officers and consultants** | Need to assess what data devices send and to whom | Policies do not show actual network behaviour, such as third-party destinations | Traffic based evidence of device destinations and behaviour to check against claims |
| **Educators and training providers** | Need repeatable, safe, current lab exercises | Datasets are static and may not reflect current devices | Reproducible lab design that can be adapted for teaching |

# 4 Existing Solutions and Gap Analysis

## 4.1 Existing Solutions

- Consumer network security products: Products such as Firewalla, Fing/Fingbox and CUJO are purpose built for home and small office users. They are easy to set up, require little technical knowledge, and present device discovery and alerts through a consumer-friendly interface, with some offering basic blocking of malicious destinations. However, their detection logic is proprietary and largely undocumented, so a flagged device gives the user little explanation of what changed or why it matters. Several depend on dedicated hardware or subscriptions, and none produces the kind of evidence, such as captures, baselines or before-and-after comparisons, that a student, administrator or consultant would need for a report or a decision.

- Open-source traffic analysis and intrusion detection tools: Wireshark supports packet capture and inspection, Zeek produces structured protocol level logs well suited to building a device baseline, and Snort provide signature-based intrusion detection. These tools are free, mature, well documented and widely used, which makes them a realistic skill set for students and small organisations to build on, and router-level capture with tcpdump makes it possible to gather this data without extra hardware. However, they are general purpose tools with a steep learning curve, and none provides an IoT specific baselining workflow out of the box. Snort is mainly signature based, so they detect known attack patterns rather than deviations from a given device's own normal behaviour.

- SIEM and log aggregation platforms: Platforms such as Splunk and the ELK stack centralise logs from many sources, support correlation rules and present results on dashboards. They scale well and are the standard tool in enterprise security operations, which makes them relevant to students preparing for SOC work. As commonly deployed, they are built for enterprise data volumes and dedicated infrastructure, and a full deployment for a handful of home devices is hard to justify. This project therefore treats the SIEM as an optional presentation layer rather than a dependency.

- Standards based behavioural control: RFC 8520 (Manufacturer Usage Description, or MUD) lets a device declare the network communications it is expected to make, so that network infrastructure can enforce a matching policy. These are principled, standards-based approaches, and MUD offers a clean way to define "normal" for a device in the device's own declared terms rather than inferring it after the fact.

However, MUD depends on manufacturer support and compatible network infrastructure that very few consumer IoT devices currently provide, making it essentially unusable for the large population of existing old devices that were never built with a MUD profile.

- IoT Inspector: An open-source desktop app that lets users visualize what their smart home IoT devices are doing behind the scenes and identify potential security and privacy violations. Run by a multi-university team led by researchers at NYU, it shows a user what their devices are doing, how many bytes they are sending and receiving, and which companies they are communicating with, so that a user can judge whether their devices are sending data when they are not supposed to, or to companies they don't know.

### 4.2 Gap Analysis

- Consumer products give visibility without explanation, and open-source tools give powerful capture and analysis without a defined IoT specific workflow. A beginner can capture traffic with Wireshark or Zeek but has no structured method for turning raw packets into a usable baseline. This affects home users, small office owners and junior practitioners most directly, and it is the core gap the project's capture and baseline workflow is designed to fill.
- SIEM platforms are built for enterprise data volumes and dedicated infrastructure. A small office administrator or home user does not need, and cannot practically support, that scale, yet has no comparably capable lightweight alternative.
- Across the solutions reviewed, none routinely measures device exposure before and after network segmentation. Segmentation is generally recommended as good practice rather than demonstrated with evidence. This affects administrators, consultants and penetration testers, who need to justify a change or confirm that a fix worked. The project addresses this directly with a before-and-after reachability and traffic comparison once IoT VLAN segmentation is applied.

# 5 Product / Project Scope

## 5.1 In scope

The project is concerned with the design, implementation and evaluation of a baseline-and-detect workflow for identifying suspicious behaviour in smart-home Internet of Things (IoT) devices. The investigation is situated within a real smart home network, on the basis that device behaviour observed under authentic operating conditions provides a more credible foundation for the project's conclusions than behaviour inferred from idealised test data alone. The environment consists of a consumer router (an ASUS RT-BE88U running Asuswrt-Merlin), which serves as both the capture point and the firewall, together with the household's own IoT devices and their controllers. No additional hardware or paid software is required.

Within this environment, the project performs passive capture of the network traffic generated by a representative set of IoT devices with different ecosystems and communication protocols, including Wi-Fi, Matter, Zigbee bridging and vendor cloud services. From this traffic, a per device communication baseline is constructed with open source traffic analysis tools (tcpdump, Zeek, tshark and dependency-free Python scripts), supported by the Pi-hole DNS log and the router's firewall logs. The baseline records each device's destinations and domains, protocols and ports, connection timing, traffic volume, and DNS and TLS metadata, and it serves as the reference against which deviations indicative of potentially suspicious behaviour are identified. Detection is performed by twelve transparent rules, each expressed as a single threshold or set test, so that the reason for every alert can be inspected and explained.

Network segmentation is also performed, with device exposure measured before and after each stage, so that any claimed benefit of segmentation is supported by comparative evidence rather than assumed based on established practice alone. The network is moved through five documented states, from the original flat configuration (State A) to a fully segmented IoT VLAN with a one-way firewall policy (State C), tuning based on test results (State D), and hardening of the router itself (State E). The effectiveness of each state is evaluated through authorised active testing from both the main network and a test host placed inside the IoT network, including simulated lateral scanning, DNS bypass, and bulk and slow data-upload scenarios. A written device function checklist confirms that the required smart home functions continue to work once the controls are in place.

To support reproducibility beyond a single household, the segmentation comparison is repeated in a simulated Mininet lab with generic IoT device types and a Linux iptables router. A Splunk pipeline and dashboard are integrated into the project to present the detection results in plain language and to rebuild the analysis independently from the raw evidence. All evidence produced by the workflow is packaged with SHA-256 fingerprints and an evidence manifest, so that findings can be independently reviewed.

## 5.2 Out of Scope

Certain areas are excluded from the scope of this project. The project does not seek to develop a deployable commercial product or application. Its output is a contribution in the form of a documented workflow and supporting evidence, not a software artefact intended for third party deployment. Detection within the proposed workflow is rule-based and is derived from explicitly defined baseline attributes. Machine learning based classification are not implemented within this project.

The project is confined to a single network environment. Enterprise scale deployment falls outside the project's scope. Formal digital forensic procedures, audit of vendor cloud infrastructure or mobile applications, and assessment of compliance against published security standards are excluded from the project's scope.

## 5.3 Assumptions and Constraints

The scope above is subject to several assumptions and constraints that qualify the interpretation of the project's findings. All data collection and experimental procedures were conducted exclusively on networks and devices under the author's own ownership or administrative authority. Certain specific findings are attributable to the router platform and device configuration employed in this study. The baseline developed in this project reflects device behaviour observed over a finite capture window. Finally, the conclusions drawn are for a small set of devices, and no claim is made that these findings show all IoT device behaviour in general.

# 6 Requirements and Success Criteria

## 6.1 Functional Requirements

FR1: The workflow must be capable of capturing network traffic passively from an authorised IoT environment, without requiring modification to the devices under observation and without disrupting their normal operation.

FR2: From the traffic captured, the workflow must be capable of constructing a per-device communication baseline. At minimum, this baseline must characterise each device's destination addresses, the ports and protocols employed, and the volume of traffic generated.

FR3: The workflow must be capable of comparing newly observed traffic against this baseline and of identifying deviations that may indicate suspicious behaviour, such comparison being conducted with reference to the specific baseline attributes.

FR4: The workflow must also support the application of network segmentation to IoT devices and must be capable of measuring device reachability and exposure both prior to and following the implementation of such segmentation.

FR5: Each stage of the workflow must produce evidence, such as capture files, device inventories and comparison results, such that findings may be independently reviewed and the procedure reproduced by a party other than the original investigator.

## 6.2 Non-Functional Requirements

NF1: The workflow must be achievable using tools that are free or available at low cost, so that it remains accessible to the home users, small business administrators and students identified as the project's primary stakeholders.

NF2: The workflow must be achievable within a time frame consistent with the operational constraints of its intended users.

NF3: It must comply with appropriate privacy and legal boundaries.

NF4: The workflow must be documented to a standard to permit replication by another practitioner working with a different device fleet.

### 6.3 Success Criteria

The criteria by which the project's success is evaluated is whether a simple baseline process can reliably distinguish normal from suspicious device behaviour, and whether network segmentation measurably reduces device exposure. The project is considered successful where the following conditions are satisfied.

First, a functioning communication baseline must be produced for a real IoT device fleet derived from traffic captured under normal operating conditions. Second, the application of network segmentation must be shown to produce at least one measurable, evidenced difference in device exposure when compared against the pre segmentation state, such that the benefit of segmentation is demonstrated rather than presumed. Third, at least one deviation detection rule must be shown to behave as expected when evaluated against a known pattern of suspicious behaviour. Lastly, the resulting documentation must be of sufficient detail and clarity that a reader could replicate the capture, baseline construction and segmentation procedures independently.

# 7 Proposed Solution and Technical Workflow

## 7.1 Overview of the Proposed Solution

The proposed solution is a staged, evidence-based workflow that transitions a given IoT device fleet from an unmonitored and unsegmented condition to a monitored and segmented one, with each transition supported by documentary evidence.

## 7.2 Stage One: Passive Traffic Capture

The workflow begins with the passive observation of network traffic at a point through which the communications of the devices under study can be observed, without requiring modification to those devices or interruption of their normal function. The objective of this stage is to obtain a sample of traffic of sufficient duration to capture each device's pattern of activity, including periodic communication with cloud infrastructure, routine local control traffic, and any other behaviour that reoccurs under normal operating conditions.

## 7.3 Stage Two: Device Inventory and Baseline Construction

The captured traffic is subsequently analysed to identify the individual devices present on the network, typically through association of traffic with a unique hardware address, and to construct from each device's traffic a profile of its characteristic communication behaviour. This profile, called the baseline, records the destinations each device contacts, the ports and protocols through which it communicates, and the volume of traffic it generates. The baseline is constructed in a form that remains legible to a nonspecialist reader such as structured logs rather than as a statistical model.

## 7.4 Stage Three: Network Segmentation

The workflow applies network segmentation to separate IoT devices from other, more sensitive network assets, employing standard network layer controls such as virtual local area network (VLAN) assignment and firewall policy.

## 7.5 Stage Four: Before-and-After Comparison

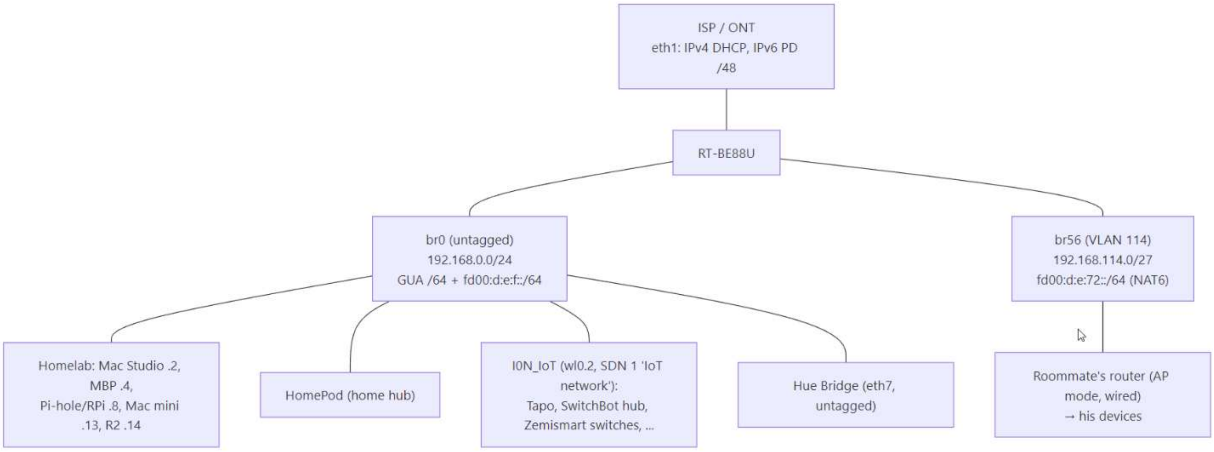
Following the implementation of segmentation, the reachability and exposure of each device is measured and compared against the corresponding measurement taken under the flat-network condition. This comparative measurement serves as the primary evidence basis for any claim made regarding the security benefit of segmentation.

### 7.6 Stage Five: Evidence Compilation and Reporting

Each of the preceding stages generates artefacts such as a traffic capture, a device inventory, a baseline profile. These artefacts are retained collectively and can achieve independent review. The intention is that each stage of the workflow, and the evidence it produces, should be capable of being reproduced by a practitioner other than the original investigator, applying the same sequence of procedures to a different device fleet.

# 8 System Architecture or Implementation Design

## 8.1 Flat network before segmentation

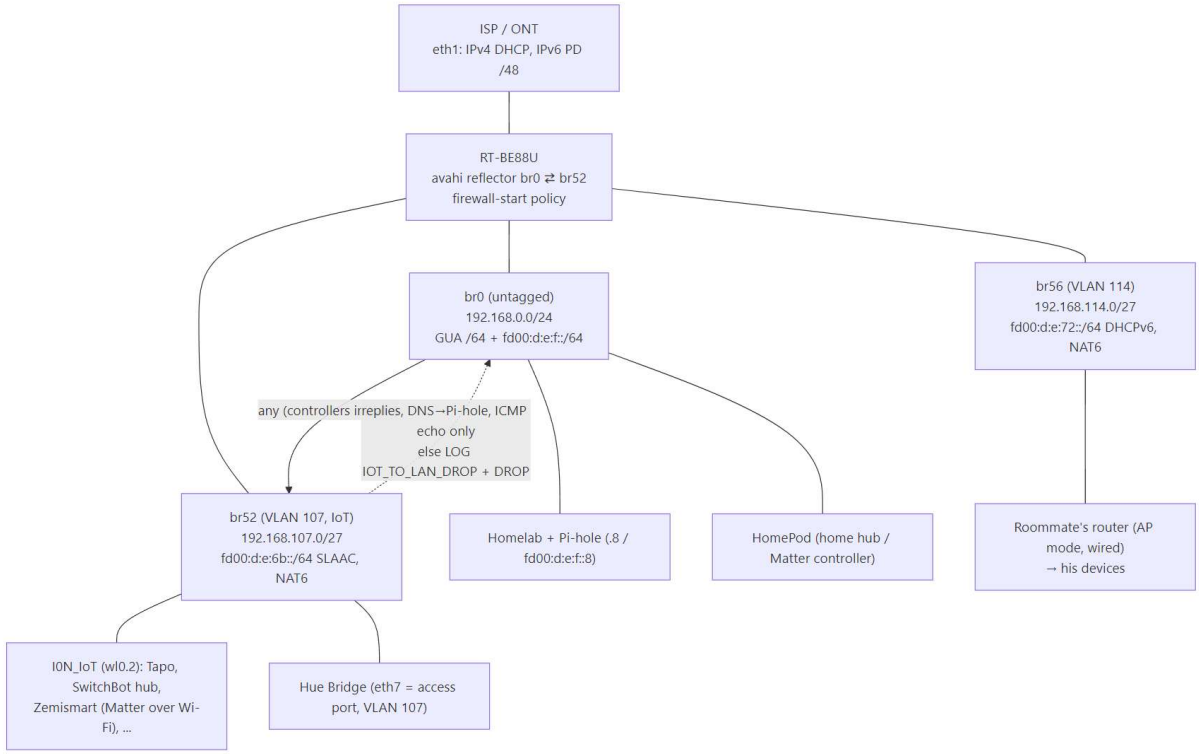



The diagram above shows the network topology as it existed prior to any segmentation of the IoT device fleet, corresponding to State A in the staged implementation. The principal components of this topology, and the role each plays in the project, are summarised below.

| Component | Role |
|---|---|
| **Router (RT-BE88U)** | Capture point, routing, firewall enforcement and service discovery reflection |
| **Main LAN** | HomePod, iPhone, Pi-hole and other trusted household hosts |
| **IoT devices (pre-segmentation)** | Wi-Fi IoT targets and the wired Hue Bridge, sharing the main LAN's broadcast domain |
| **DNS service** | The approved resolver (Pi-hole) and the device DNS evidence it records |
| **Roommate network (VLAN 114)** | Hosts explicitly excluded from active testing and analysis |

In this state, the router receives WAN connectivity from the ISP's optical network terminal on interface eth1, by IPv4 DHCP and IPv6 prefix delegation of a /48. A single untagged bridge, br0, carries the entire main LAN on 192.168.0.0/24, with a globally routable IPv6 /64 and a locally scoped /64 (fd00:d:e:f::/64) addressed to it. This bridge carries four categories of host without any logical separation between them: the household's general-purpose and homelab machines, the HomePod acting as home hub, the IoT Wi-Fi network, and the wired Hue Bridge on interface eth7. Because all these sit on the same bridge and the same IPv4 subnet, every IoT device is adjacent to every trusted household host. Therefore, any device on br0 can address, discover and attempt to reach any other device on br0. The only pre-existing segmentation in this topology is a separate bridge, br56, carrying VLAN 114 on 192.168.114.0/27 with its own NAT66-translated IPv6 range (fd00:d:e:72::/64), which connects the roommate's own router.

## 8.2 Segmented Network (State C)



The diagram above shows the topology after the fully segmented configuration, State C, was implemented. The router's role expands in this state to include two additional functions alongside its existing routing and firewall duties: an avahi service discovery reflector operating between br0 and the new IoT bridge, and the firewall-start policy that enforces the segmentation boundary between them.

The main LAN bridge, br0, is retained unchanged in addressing but is now reduced to the household's homelab and Pi-hole hosts and the HomePod. The IoT devices are moved to a new, dedicated bridge, br52, carrying VLAN 107 on the private range 192.168.107.0/27, with IPv6 connectivity provided through SLAAC over a locally-scoped fd00:d:e:6b::/64 prefix and NAT66 for outbound IPv6 translation. Both the Wi-Fi IoT fleet, reached over wl0.2 and comprising the Tapo plug, the SwitchBot hub and the Zemismart Matter-over-Wi-Fi switches, and the wired Hue Bridge are placed on this bridge. The Hue Bridge's port on eth7 is reconfigured as a VLAN 107 access port so that its traffic is carried on the IoT bridge despite remaining a physical, wired connection. The roommate's VLAN 114 bridge, br56, is retained without modification alongside this change.

The boundary between br0 and br52 is governed by the one-way policy implemented in firewall-start. Traffic initiated by the main-LAN controllers toward the IoT bridge is accepted, while traffic in the reverse direction is accepted only where it constitutes a reply to an already-permitted connection, a DNS query directed to Pi-hole, or an ICMP echo. Every other IoT-initiated attempt to reach the main LAN is both logged, under the IOT_TO_LAN_DROP rule and dropped. Because this firewall boundary removes the adjacency that existed in the flat topology, service discovery protocols that depend on local multicast, required for HomeKit and Matter device discovery, would otherwise stop functioning across the new boundary. The avahi reflector shown between br0 and br52 exists specifically to re emit the relevant multicast traffic across the segment boundary without otherwise weakening the firewall policy.

# 9 Security, Privacy, and Risk Analysis

## 9.1 Privacy Risk to the Household

Network metadata, even where no payload is inspected, can reveal a considerable amount about household activity. The periodicity and timing of DNS queries and device check ins were sufficient to reconstruct when an occupant was actively interacting with a given smart-home application, and this reconstruction was used in a controlled scenario to validate that the project's detection rules behaved as expected. The same capability that supports legitimate detection is also a capability to infer household behaviour. Analysis was confined to metadata the project genuinely needed for its stated research questions, and no attempt was made to extend inference beyond what the baseline-and-detect workflow required.

## 9.2 Risk to the Availability and Integrity of the Environment Under Test

Passive traffic capture, as practised throughout the baselining stage of this project, carries negligible risk to the availability of the devices or network under study, since it does not alter traffic in transit. However, active testing including the port and service scans used to establish the exposure of IoT devices and the router itself, carries a higher risk of disrupting normal operation. This risk was managed by confining active testing to defined, short time windows, bracketed by router counter and connection-state snapshots taken immediately before and after each test, so that any disruption to the household network would be both visible and attributable to a specific test rather than discovered only indirectly. When a given test produced an unexpected result, such as the discovery of previously unrecognised traffic from devices external to the project's own inventory, the before and after snapshots made it possible to attribute the traffic correctly rather than treating it as an artefact of the test itself.

## 9.3 Risks Identified in the Devices Under Study

Two findings produced during this project constitute confidentiality or integrity weaknesses in commercial devices. The first is a firmware defect in a hub device that causes it to transmit Matter protocol identifiers to the public internet as a side effect of a malformed response to multicast discovery queries. This was confirmed by validating inbound error messages from infrastructure outside the household network, but the project did not attempt to determine what further information an attacker could extract from the exposed identifiers, judging that question to be outside the scope. The second is a bridge device that transmits a persistent authentication adjacent token over unencrypted HTTP. The project confirmed that the token does not vary between requests, which is sufficient to establish the weakness, but did not attempt to intercept, replay, or otherwise

make use of the token, since doing so would constitute exploitation rather than observation.

### 9.4 Evidence Handling

Because the project's raw evidence was subsequently packaged for review by others, the handling of that evidence after collection constitutes a privacy risk. Before any capture, log, or derived artefact was shared outside the author's own working environment, it was processed to remove or replace specific sensitive elements, the household's public internet address and delegated address prefix were consistently replaced with placeholder values across every file in which they appeared, the persistent token was overwritten, and all identifying material belonging to the third-party network segment was removed outright. This was done in a manner that preserved the technical validity of the evidence. Evidence that is fit to analyse is not automatically fit to share, and the obligation to protect sensitive detail does not end once a finding has been produced, but persists through however many hands the supporting evidence subsequently passes.

### 9.5 Authorisation Boundary and Third-Party Risk

A second occupant of the household under study maintains a separate network segment for personal devices, and this segment falls outside the project's administrative authority. Frames belonging to that segment were excluded from every capture and every subsequent stage of analysis, that segment was never an active scanning target, and firewall rules intended to constrain it were documented and implemented without being exercised, since doing so would have required probing a network the project is not authorised to test. This boundary illustrates a general principle that physical or logical adjacency to a third party's devices does not authorise observing or testing them, and a sound risk analysis treats that boundary as fixed even where it limits what can be verified.

# 10 Development Progress and Evidence

## 10.1 Capture Script and Methodology

```
iot-capture.sh - timed packet capture for the IoT segment (Asuswrt-Merlin, HND)

Captures each interface with its own tcpdump process (keeps the Ethernet/MAC
headers; avoids the Linux "cooked" header that -i any produces), optionally
rotates output files, and stops after a configurable duration. Broadcom flow
cache (fc) and Runner offload are disabled for the capture window so offloaded
flows stay visible, then re-enabled on exit, including on early stop.

Usage:
  iot-capture.sh [-b] [-d DURATION] [-r ROTATE] [-i "IF1 IF2"] [-o BASEDIR] [-f BPF] [-n]

  -b           run in background: detach into a new session (setsid), no
               controlling terminal, survives SSH logout. Replaces nohup.
  -d DURATION  total capture time; accepts s/m/h/d suffix (default: 24h)
               e.g. 90m, 6h, 2d, 3600 (bare number = seconds)
  -r ROTATE    file rotation interval, same format (default: no rotation,
               one file per interface for the whole run)
  -i IFACES    space-separated interface list (default: "wl0.2 eth7")
  -o BASEDIR   capture storage root (default: /tmp/mnt/opt/captures)
  -f BPF       optional BPF filter applied to all interfaces
  -n           do NOT toggle fc/runner offload (capture may miss offloaded flows)
```

The capture script, iot-capture.sh, was developed to run on the router itself rather than on an external capture device. It invokes tcpdump independently on each interface of interest, disables the router's hardware flow offload mechanism before capture begins, restores it once capture is complete, and writes a per interface run log recording packet and drop counts throughout. This script constitutes the foundation of every other piece of evidence in this section, since the validity of the baseline, the inventory and the segmentation evaluation all depend on the completeness and integrity of the underlying capture. The decision to disable flow offload during capture, rather than leaving it enabled, follows directly from a troubleshooting finding reported below. It is included here first because it had to be resolved before any of the capture evidence that follows could be trusted. The remaining limitation at this stage is that the script addresses offload-related invisibility but, as discussed in 10.6, does not address a separate, unresolved blind spot concerning device to device traffic bridged within the wireless driver.

### 10.2 Baseline Capture Run Log

A 24 hour baseline capture was executed using the script described above, across two interfaces: the dedicated IoT wireless virtual access point (wl0.2) and the wired switch port serving the Hue Bridge (eth7). The run log for this capture records 858,199 packets captured on wl0.2 and 958,030 packets on eth7, with a drop count of zero on both interfaces.

### 10.3 Device Inventory

Analysis of the baseline capture identified 31 distinct unicast MAC addresses on the network. Of these, eight were confirmed, through a combination of vendor OUI lookup and manual cross-referencing against each physical device. The device fleet consists of the Hue Bridge, the SwitchBot Hub 2, the Tapo smart plug, the two Zemismart Matter switches, the HomePod 2, and the Raspberry Pi running Homebridge and Pi-hole. The Eve Room sensor was additionally identified as reachable only indirectly, via the HomePod's Thread border router function, rather than appearing as an independent address on the capture interfaces. A further device, associated with a separate household VLAN, was identified but excluded from analysis.

### 10.4 Segmentation Configuration

Five progressive network states were implemented, tested and documented. Each state was tested before the next was built, and each later state responds to a problem found by testing the one before it.

- State A, flat: The original configuration. IoT devices share the main LAN bridge (br0), its IPv4 subnet and its broadcast domain, and hold globally routable IPv6 addresses.

- State B, VLAN created in the web interface: The IoT Wi-Fi is moved to VLAN 107 using the router's built-in option. There is no IPv6 and no inter-segment policy yet, and HomeKit and Matter device discovery breaks.

- State C, fully segmented: A dedicated /27 subnet on a new bridge (br52), with the Hue Bridge's wired port as a VLAN 107 access port. IPv6 uses ULA addressing, SLAAC and NAT6. A one-way firewall policy lets the main LAN start connections into the IoT segment and drops and logs everything the IoT side starts towards the main LAN (IOT_TO_LAN_DROP), apart from replies, DNS to Pi-hole and ping. An avahi mDNS reflector carries device-discovery traffic across the boundary.

- State D, fixes found by testing: The UDP connection-tracking timeout is raised for Matter reports (10.5). IoT-to-WAN connections are logged. NTP is served on each VLAN's own gateway. The reflector is restricted to HomeKit, Matter and Hue service types, which cut the service types answered across the boundary from 38 to 5–7. VLAN IPv6 addresses and STP settings are now restored automatically after the firmware rebuilds the bridges.

- State E, router hardening: Testing from a host inside the IoT network showed that state D contained the hosts but not the router itself. Traffic addressed to the router passes through its INPUT chain, which the FORWARD policy of state C does not cover. From the IoT network, SSH, the router's own DNS resolver, a DLNA media server and reverse-proxy ports were all reachable, and a device could bypass Pi-hole by querying an internet resolver directly. State E allows only DHCP, NTP, mDNS and ICMP to the router from the IoT network and drops SSH on every VLAN. It redirects all DNS to Pi-hole over IPv4 and IPv6, UDP and TCP, drops DNS-over-TLS, and blocks the fibre modem's management page from every VLAN except the main LAN.

### 10.5 Troubleshooting Record: Connection-Tracking False Positive

Following the transition to State C, firewall logs showed repeated IOT_TO_LAN_DROP entries for UDP traffic on source port 5540, originating from the Zemismart Matter switches and directed at the HomePod. Because this traffic was legitimate Matter subscription-report traffic rather than an actual policy violation, it was investigated as a false positive. The command below was used to inspect the router's active connection-tracking table and confirm that entries for this flow were expiring between reports:

*cat /proc/net/nf_conntrack | grep <device_ip>*

This showed that the router's default UDP "assured" connection-tracking timeout, 180 seconds, was shorter than the interval between successive Matter subscription reports from the affected devices. Because the connection-tracking entry expired before the next report arrived, each report was treated by the firewall as the start of a new, and therefore disallowed, connection rather than as a continuation of an already-permitted one. The fix applied was to increase the relevant timeout in the router's non-volatile memory:

*nvram set ct_udp_timeout="30 1800"*
*nvram commit*

This finding matters for two reasons. First, it shows that the segmentation and logging architecture worked correctly. The false positive came from a timing interaction with the firewall's state tracking, not from a failure of the policy itself. Second, it illustrates a general class of problem beyond this router. Stateful firewalls that segment IoT devices are liable to misclassify legitimate protocol traffic as policy violations unless the connection tracking timeouts match the device's actual reporting interval. The fix required no new firewall exception as to not weaken the boundary to suit the device.

The fix has since been validated over a longer period. Before it, 60 IOT_TO_LAN_DROP entries from four devices were logged in 35 minutes. After it, the router showed Matter sessions with remaining lifetimes of 892-1,674 seconds, consistent with the new 1,800-second timeout, and zero drops on both IPv4 and IPv6. During the final function check on 2 November, with the Home app closed for more than five minutes, no port-5540 reports were dropped in that test's window. One related pattern remains. Straight after a router reboot, the empty connection table causes a few short bursts of the same drops (25 entries in total, each burst lasting 5-17 seconds) until the controllers refresh their sessions. These bursts had no effect on device function, and the dashboard labels them as information rather than alerts.

### 10.6 Troubleshooting: Hardware Flow Offload and Capture Visibility

During early capture testing, a discrepancy was identified between traffic known to be occurring, based on device behaviour and the router's own connection-state table, and traffic visible in tcpdump output and firewall logs. Investigation traced this to the router's hardware flow offload feature (the Broadcom flow cache/runner), which handles established network flows directly in hardware once a flow has been classified, bypassing the kernel networking stack that tcpdump and netfilter logging both depend on for visibility. The effect is that once a flow is offloaded, it becomes invisible to both capture and logging, even though it continues to be forwarded and remains subject to enforcement.

This was confirmed by comparing capture output for the same device traffic with offload left enabled against capture output with offload explicitly disabled beforehand, the latter showing substantially more visible traffic for the same devices and time window. The fix, incorporated into iot-capture.sh, disables offload at the start of each capture run and restores it afterwards via a marker file, so that the router's normal forwarding performance is unaffected outside of capture windows.

# 11 Testing, Validation, and Evaluation

## 11.1 Evaluation Approach

The evaluation was based on three questions: could the workflow provide a useful baseline, did it detect any atypical device actions, and did it indicate if there was any network exposure reduction due to segmentation? To address these questions, the traffic from the flat network was compared to traffic that was collected after segmentation. These captures were for approximately 24 hours each. Authorised connection tests, router logs and configuration records were also used to check the findings. However, there were some changes to some of these settings during the segmented capture, and the results are of what happened in this network — not a controlled experiment.

## 11.2 Capture and Baseline Validation

The baseline capture saw 858, 199 packets on wl0.2 and 958, 030 packets on eth7. There were no reported kernel drops on either interface. These numbers are useful in determining if collection was as expected, however, a zero count does not indicate that all traffic on the network was collected. Some wireless traffic may go between devices without being caught by the capture interface. Disabling hardware flow offload made it easier to see, run logs documented the collection process, and the use of the SHA-256 checksums enabled later integrity checks.

A baseline was established for the information required to describe each directly observed IoT device – destinations, ports, protocols and traffic volumes. A comparison of some results was done between the tools. If you look at the flat capture, for example, and the segmented capture, you can see that 935 Hue DNS queries were recorded with packet analysis and 1, 966 with Pi-hole. Such is the case for this agreement as it supports those DNS counts, while there are other fields that still require their own checks. To minimize double counting, traffic from devices to the Hue was analysed on the wired traffic and on the wireless traffic.

## 11.3 Detection and Mitigation Results

In the authorised tests, simulated connection attempts from the IoT segment towards the main LAN showed up in the firewall logs. This revealed that such an attempt could be detected by the monitoring rule that was tested. There was also unanticipated traffic for SwitchBot, destined for a reserved address. This behaviour was also confirmed by WAN observations and upstream ICMP responses. These findings would provide the

team with certain events to investigate; however, if there were unusual traffic on its own, it would not necessarily mean a device was compromised.

The Matter issue highlighted the importance of device behaviour checks against alerts. The default 180 seconds assured connection-tracking timeout period was expiring between legitimate UDP subscription reports, causing them to be dropped. Reviewing the firewall logs as well as connection-tracking information helped clarify the issue. The observed drops were resolved by increasing the timeout to 1,800 seconds without the need to add a wide firewall exception. An extended follow up test is still required. This should show a comparison of the number of drops over comparable timeframes and verify that device reports are still being received.

Additionally, the captures indicated reduced visibility of the shared network after segmentation. The number of different layer-2 senders decreased from 26 to 8 on wl0.2, and 23 to 7 on eth7. There were 14 mDNS senders visible on the IoT side that reduced to one reflector. These are helpful indicators of the change in exposure, but do not provide a percentage reduction in overall security risk. The possibility of traffic passing by the reflector and the connections allowed by the firewall must still be taken into account.

### 11.4 Metrics and Remaining Evaluation

The main measures for the evaluation are completeness at baseline, detection results, blocked connections and device functionality and reproducibility. Baseline completeness is checking if the desired information is present for each of the target devices. The following measurements will apply for the rest of tests:

- Scenario detection rate: number of detected suspicious test scenarios/number of the tested suspicious test scenarios.
- Containment pass rate: number of blocked prohibited paths / number of prohibited paths tested (IPv4 and IPv6).
- Functionality: if local control, routine reporting, updates, restart recovery and remote access are still functional.

Reproducibility: can another practitioner follow the directions and get similar results?
The false alarm should be measured on a different normal traffic. The traffic of each device should be split into the same time windows. Then, false positive rate is computed as number of normal windows with a false alarm/total number of normal windows. A log of activities is required to determine whether a certain event is legitimate; for example, if a piece of firmware is downloaded or a new service is connected.
There is no overall accuracy, false-positive or false-negative rate since a complete labelled dataset is not available. The controlled large-volume upload test as well as the full device-

function checklist and independent repeat run are also not completed. The outputs obtained so far indicate that the workflow can be employed to study the network, though further tests are required to evaluate the detection capabilities of the workflow.

# 12 Deployment, Operation, and Maintenance Considerations

## 12.1 Deployment Environment and Permissions

The workflow could be used by a home user with networking experience or a small-office administrator. It requires a router capable of doing VLANs, to change the firewall, to capture packets, and to have a computer for analysis. The current setup uses Asuswrt-Merlin settings. The administrator would need to verify the names of the interfaces, as well as the startup scripts and the firewall commands before applying it to other routers. To capture traffic and change network settings, administrative access and permission from the network owner is required.

## 12.2 Deployment and Recovery Procedure

It is the administrator's responsibility to list the devices and services needed before changing, and to make a backup copy of the router configuration. Prior to migration of a small group of devices into the IoT segment, a baseline should be collected. Next, test firewall rules in both directions with IPv4 and IPv6 and then test discovery and app control/internet access. The current policy allows connections from the main LAN towards IoT devices. A user of a compromised main-LAN host could thus still gain access to them. Access to certain controllers would need to be restricted with additional configuration and testing.

The change of network should only be made in a planned maintenance window and there should be a clear method to revert back to the original network configuration. An incorrect entry may cause a DNS failure, as well as prevent device discovery or router administration. Another test to try is restarting the router as some settings might not take effect after restart. Next, the administrator will need to verify VLAN membership, firewall rules, IPv6 addresses and device operation. Hardware offload should also be re-enabled after capture as it may impact the performance of the router if it is not enabled.

## 12.3 Monitoring and Maintenance

Typical tests include failed captures, available storage, new destinations, blocked IoT-to-LAN connection, and repeated DNS or connection errors. A baseline assessment should be performed whenever there is a change in the devices' firmware or a new device

is added. Traffic should be investigated before it is declared as accepted normal traffic; otherwise an abnormal connection that persists could become part of the normal traffic. Version control of configuration files, scripts and rules helps in understanding what configuration were in effect when an alert was raised.

Restrict access to the captures and logs and define a retention period before collecting. The packaged Splunk app and guide document event preparation, ingestion and the dashboard checks. When replacing an event batch, clear the earlier copy or use a separate test index so the dashboard does not count duplicate events.

### 12.4 Documentation and Operational Feasibility

The pipeline guide documents the prerequisites, site configuration, ingestion and dashboard use. The Mininet README covers the separate lab. The tools reduce the need for dedicated monitoring hardware, but deploying the workflow still requires router administration and basic knowledge of VLANs, IP addressing and packet captures. We did not conduct a formal usability evaluation with novice operators.

# 13 Limitations and Future Improvement

## 13.1 Dataset and Generalisation

The field study is limited to one household, one router platform and a small set of the devices. The bulk observation periods are approximately a day each state. Clouds can also change their behaviour, and short captures do not track those changes or infrequent firmware updates. The device models in the Mininet lab are based on IPv4 and the evidence of vendor behaviour/ IPv6 is from household study.

## 13.2 Visibility and Experimental Constraints

You cannot see traffic using the wireless driver even if flow offload is off. An additional follow-up run also had a portion of its wireless window observation lost and adding to this, the log rotation of the firewall removed the initial part of a wireless upload test and stored it in the log file. Corroborating the upload we used packet captures and router counters.

Activity and configurations of the firmwares varied between the original capture windows. This means the compared network depicts observed changes of this network. It cannot differentiate one factor as the sole contributor - segmentation. Because of the privacy redactions, the shared "whole-network" captures won't be able to reproduce all the original whole-network counts.

## 13.3 Detection and Usability Limitations

Baseline and thresholds are used to determine "rule matches". Any legitimate changes may trigger alerts, while suspicious traffic may appear like normal traffic. A follow-up evaluation should be conducted to ahead mark the normal activity, measure false alerts using the time windows held back for this purpose, and ask a second operator to repeat an old tested activity (similar to case #6). The current replay/cross-tool agiometry quantifies a different aspect of the reproducibility.

## 13.4 Reflection on the Workflow

The Matter investigation was an obvious place where we wanted to maintain a cohesive data set of packet, firewall and function evidence. The Log properly recorded the drop. The device behaviour and connection table provided reasons for the packet being a legitimate packet. A second problem with monitoring and relying on one metric (the volume of upload any specific device generated) was that neither the per-device rule nor the number of connections conveyed any specific information.

# 14 Conclusion

This project provided a low cost option to set device baselines and set up a method to measure variation and test an IoT segmentation in an authorized home. Five device profiles found transparent rules, finding lateral scanning and both versions of the upload test and bypassing the DNS. The Splunk dashboard provides a means to trace back from alerts to their evidence.

Finally, all the docked unauthorised paths from the IoT network were blocked while the passport tests were passed on all the tested device-functions, with a total of 28 documented. A separate exploration of the difference that might have been introduced by segmentation and router hardening is done with the Mininet comparison.

The evidence to support the workflow for the network and scenarios tested is present. Extended monitoring and a test with real-time (held-out) normal traffic would be useful to determine the frequency of false warnings.

# 15 References and Appendices


Affinito, A., Zinno, S., Stanco, G., Botta, A., & Ventre, G. (2023). The evolution of Mirai botnet scans over a six-year period. *Journal of Information Security and Applications, 79*, 103629. https://doi.org/10.1016/j.jisa.2023.103629

Antonakakis, M., April, T., Bailey, M., Bernhard, M., Bursztein, E., Cochran, J., Durumeric, Z., Halderman, J. A., Invernizzi, L., Kallitsis, M., Kumar, D., Lever, C., Ma, Z., Mason, J., Menscher, D., Seaman, C., Sullivan, N., Thomas, K., & Zhou, Y. (2017). Understanding the Mirai botnet. In *Proceedings of the 26th USENIX Security Symposium* (pp. 1093–1110). USENIX Association. https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/antonakakis

Asuswrt-Merlin. (n.d.). *Asuswrt-Merlin project website*. https://www.asuswrt-merlin.net/

Cisco. (n.d.). *Snort: Network intrusion detection and prevention system*. https://www.snort.org/

CUJO AI. (n.d.). *CUJO AI*. https://cujo.com/

Fagan, M., Megas, K. N., Scarfone, K., & Smith, M. (2020). *IoT device cybersecurity capability core baseline* (NISTIR 8259A). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.IR.8259A

Fing. (2016, November 21). *Fingbox affordable home network security device launches*. https://www.fing.com/news/affordable-home-network-security-device-launches/

Firewalla. (n.d.). *Firewalla Gold*. https://firewalla.com/products/firewalla-gold

FoxIO. (n.d.). *JA4+ network fingerprinting* [Computer software]. GitHub. https://github.com/FoxIO-LLC/ja4

Huang, D. Y., Apthorpe, N., Li, F., Acar, G., & Feamster, N. (2020). IoT Inspector: Crowdsourcing labeled network traffic from smart home devices at scale. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies, 4*(2), Article 46. https://doi.org/10.1145/3397333

Lantz, B., Heller, B., & McKeown, N. (2010). A network in a laptop: Rapid prototyping for software-defined networks. In *Proceedings of the 9th ACM Workshop on Hot Topics in Networks* (Article 19). ACM. https://doi.org/10.1145/1868447.1868466

Lear, E., Droms, R., & Romascanu, D. (2019). *Manufacturer Usage Description specification* (RFC 8520). Internet Engineering Task Force. https://doi.org/10.17487/RFC8520

Meidan, Y., Bohadana, M., Mathov, Y., Mirsky, Y., Shabtai, A., Breitenbacher, D., & Elovici, Y. (2018). N-BaIoT—Network-based detection of IoT botnet attacks using deep autoencoders. *IEEE Pervasive Computing, 17*(3), 12–22. https://doi.org/10.1109/MPRV.2018.03367731

Neto, E. C. P., Dadkhah, S., Ferreira, R., Zohourian, A., Lu, R., & Ghorbani, A. A. (2023). CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment. *Sensors, 23*(13), 5941. https://doi.org/10.3390/s23135941

OWASP Foundation. (2018). *OWASP Internet of Things Top 10 2018*. https://owasp.org/www-project-internet-of-things/

Open Information Security Foundation. (n.d.). *Suricata*. https://suricata.io/

Pi-hole. (n.d.). *Pi-hole documentation* https://docs.pi-hole.net/

Shi, Z., Jin, W., & Ma, R. (2026). *Challenge Studio: Detecting suspicious behaviour in smart-home IoT devices* [Source code, evidence and results]. GitHub. https://github.com/XuYuanLah/Challenge-Studio-

Splunk. (n.d.). *Set up and use HTTP Event Collector in Splunk Web*. https://help.splunk.com/en/splunk-enterprise/get-data-in/collect-http-event-data/set-up-and-use-http-event-collector-in-splunk-web

The Tcpdump Group. (n.d.). *Tcpdump and libpcap*. https://www.tcpdump.org/

Wireshark Foundation. (n.d.). *tshark manual page*. https://www.wireshark.org/docs/man-pages/tshark.html

Zeek Project. (n.d.). *The Zeek network security monitor*. https://zeek.org/

| Appendix | Content | Location |
|---|---|---|
| **A** | Evidence manifest (E01–E119), SHA-256 fingerprints of every file in the evidence bundle, and provenance | 03-Result/IoT-Segmentation/evidence_manifest.md, SHA256SUMS, PROVENANCE.tsv |
| **B** | Results-to-evidence map: each result, its deciding numbers, evidence IDs and explaining section, with a glossary | 03-Result/IoT-Segmentation/results_to_evidence.md |
| **C** | Full analysis, including baselines, all findings, the R1–R12 rule set (§8) and the evaluation against the Learning Contract metrics (§9) | 03-Result/IoT-Segmentation/analysis.md |
| **D** | Segmentation report covering states A–E, and the version-controlled router configuration, including the capture script iot-capture.sh | 03-Result/IoT-Segmentation/segmentation_report.md; 02-Code/IoT-Segmentation/asuswrt-configs/ |
| **E** | Test plans and the completed 28-point function checklist, with screenshots | 03-Result/IoT-Segmentation/planning/; 03-Result/IoT-Segmentation/function_check/ |
| **F** | Active test outputs (nmap results for states D and E), firewall logs and router counter snapshots | 03-Result/IoT-Segmentation/iot_active/, iot_retest/, firewall_logs/, router_states/ |
| **G** | Sanitised packet captures for each recording window, and derived flow summaries | 03-Result/IoT-Segmentation/20261006-215327_flat/ to 20261031-184353_vfd/; derived/ |
| **H** | Splunk pipeline, Splunk app and dashboard guide, with setup instructions and dashboard screenshots | 02-Code/IoT-Segmentation/pipeline_guide.md; 02-Code/IoT-Segmentation/pipeline/ |
| **I** | Mininet simulated lab: code, setup instructions and the flat, segmented and hardened results | 02-Code/mininet iot lab/; 03-Result/Mininet simulated lab results/comparison.json |
| **J** | Peer review received and the team's response | 06-Peer Review/ |